

Теория формальных языков

Лабораторная работа №2

Вариант 8

Работу выполнила:

Студент группы ИУ9-51Б

Григорьева Софья

Содержание

1	Задание	2
2	ДКА	4
3	НКА	5
4	ПКА	6
5	Расширенное регулярное выражение	7
6	Тестирование	8

1 Задание

Дано регулярное выражение:

$$((aa|ba)(ab)^*(bb)^*)^*$$

По имеющемуся академическому регулярному выражению построить:

- Минимальный ДКА, распознающий его язык (минимальность обосновать таблицей классов эквивалентности)
- Возможно малый НКА, распознающий его язык. Возможно малый переключающийся (с конъюнкцией) КА, распознающий его язык. Частично обосновать таблицами множеств классов эквивалентности.
- Расширенное регулярное выражение, распознающее тот же язык. В расширенном выражении можно использовать:
 - wildcard-операцию $.$ для замены произвольного символа алфавита;
 - положительную итерацию τ^+ и опцию $\tau?$, где $\tau^+ = \tau\tau^*$, $\tau? = (\tau|\varepsilon)$;
 - операции предпросмотра $\tau_0(? = \tau_1)\tau_2 \equiv \tau_0((\tau_1.*) \cap \tau_2)$ и ретроспективной проверки $\tau_0(? \leq \tau_1)\tau_2 \equiv (\tau_0 \cap (\tau_1.*))\tau_2$, а также их отрицательные версии $\tau_0(?!\tau_1)\tau_2 \equiv \tau_0((\tau_1.*) \cap \tau_2)$ и $\tau_0(? < !\tau_1)\tau_2 \equiv (\tau_0 \cap (\tau_1.*))\tau_2$;
 - классы букв $[c_1 \dots c_k] \equiv (c_1|c_2|\dots|c_k)$ и их дополнения $[\hat{c}_1 \dots \hat{c}_k]$;
 - (обязательно) маркеры начала и конца выражения $\wedge$ и $\$$.

Все конструкции выше можно делать вручную. Для их фиксации желательно использовать формат md со встроенными latex-формулами, поддерживаемыми MathJax, либо формат md обсидиана (он допускает расширенную поддержку latex в формулах). Можно и чистый latex. Допустимо сдавать и фото на листочках, но за оформление в Markdown/Latex добавляется 1 балл.

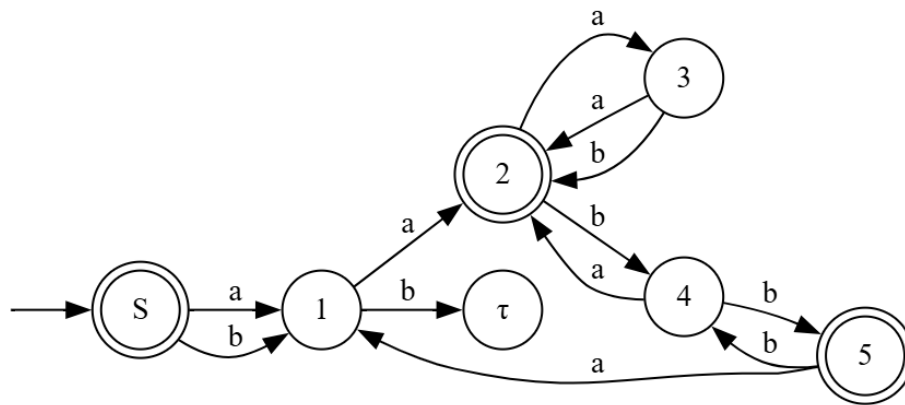
Провести автоматическое тестирование предполагаемой эквивалентности построенных распознавателей. Тем самым необходимо построить алгоритмы, определяющие принадлежность слова языку академического регулярного выражения, ДКА, НКА и ПКА.

Эквивалентность расширенного регулярного выражения тестировать не обязательно, но если сделаете распознаватель для таких выражений,

то будет добавлен дополнительный балл. Если не делаете, то для расширенного регулярного выражения достаточно описать содержательно (полуформально), почему оно должно распознавать тот же самый язык.

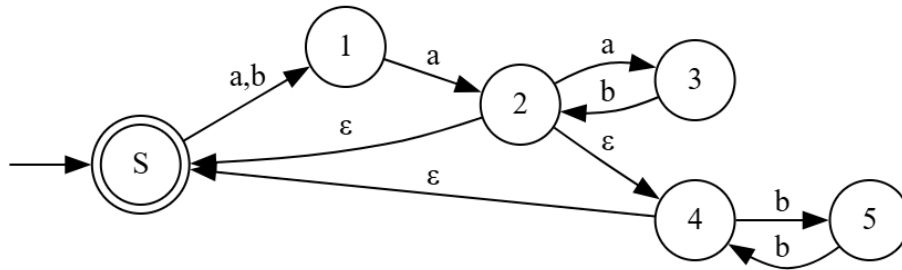
Требуется только фазз-тестирование эквивалентности: строится случайное слово ω и проверяется, принадлежит ли он языкам регулярного выражения, ДКА, НКА и ПКА согласованно.

2 ДКА



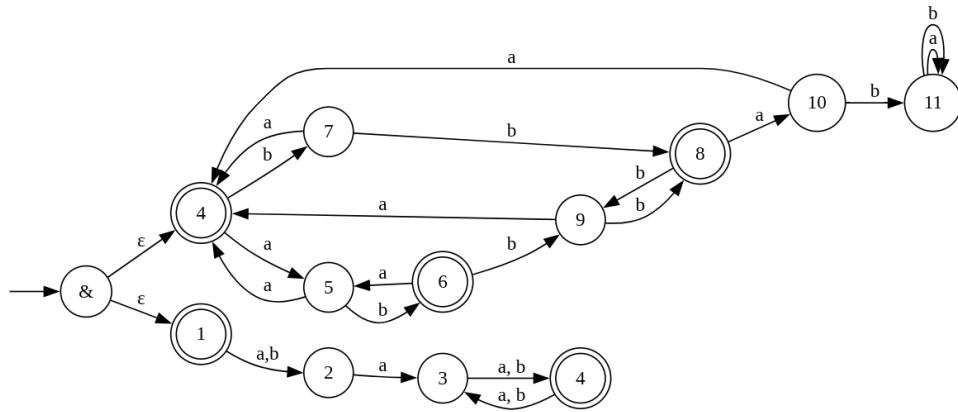
	bb	babbb	e	b	ab
e	0	0	1	0	0
a	0	1	0	0	0
aa	1	0	1	0	1
aaa	0	1	0	1	0
aab	0	0	0	1	0
aabb	1	0	1	0	0
ab	0	0	0	0	0

3 HKA



	ab	bb	babbb	aa	b	aaa
aa	1	1	0	1	0	0
aabb	0	1	0	1	0	0
aaa	0	0	1	0	1	1
e	0	0	0	1	0	0
aab	0	0	0	0	1	1
a	0	0	0	0	0	1

4 ПКА



	b	bab	aaa
aaa	0	1	1
b	0	0	1
bbab	0	0	0

5 Расширенное регулярное выражение

$\wedge (.a(ab)^*(bb)^*)^*\$$

6 Тестирование

Исходный код программы представлен в файле «fuzzTest.erl».

Программа проверяет эквивалентность трёх типов автоматов (ДКА, НКА, ПКА) и регулярного выражения через фазз-тесты на случайных словах.

В программе реализованы следующие основные функции:

- **ДКА:**
 - `dka()` – таблица переходов;
 - `dkaFinals()` – финальные состояния;
 - `useDka(Word)` – проверка слова.
- **НКА:**
 - `nkaTransA()`, `nkaTransB()`, `nkaTransEps()` – таблицы переходов;
 - `nkaFinals()`, `nkaStart()` – финальные и стартовое состояния;
 - `useNka(Word)` – проверка слова с учётом ε -переходов.
- **ПКА:**
 - `pkaTrans()` – таблица переходов;
 - `pkaFinals()`, `pkaStartStates()` – финальные и стартовые состояния;
 - `usePka(Word)` – проверка слова для всех стартовых состояний.
- **Генерация слов:**
 - `generateRandomWord(Length)` – случайное слово из символов 'a' и 'b';
 - `randomLetter()` – случайный выбор символа.
- **Основной цикл тестирования:**
 - `main()` – выполняет фазз-тесты для слов длиной 1–30;
 - `mainLoop(Length, Max, N, Regex)` – генерация слов, проверка через ДКА, НКА, ПКА и регулярное выражение, сравнение результатов и вывод ошибок.